Dependabot: a gentle trickle of updates you can trust

A few PRs a month, never a flood, and nothing reaches the live site untested.

Walkthrough dependabot | Changed 2026-11-04 | For Whoever keeps a site's dependencies

You are here: drasticstatic lab › `my-template` › walkthroughs › `DEPENDABOT.md` · public · `main` is the working branch

TL;DR

Dependabot keeps a site's dependencies current and patched. Kept gentle, it's a few PRs a month, never a flood. Kept trustworthy, nothing reaches the live site untested.

1. **A monthly config:** minor and patch updates in one PR, majors in a second, Actions in a third.
2. **Security fixes don't wait for the schedule.** With alerts on (see Org security settings), they open as soon as a fix exists.
3. **Test every PR** in its own copy, against a baseline, before merging.
4. **Merge one at a time,** waiting for each deploy and checking a real stylesheet loads.
5. **Hold a risky major** with an `ignore` rule, and write the reason beside it.

Learned on 2026-10-26, rolling Dependabot out across a foundation's two sites and several friendly communities' AstroLogs, portals, a Discord helper bot and a web3 dApp. The foundation's 11 updates were tested and rehearsed one by one. Elsewhere 32 PRs were tested: 21 green, 10 held for a reason, 1 obsolete.

The config

`.github/dependabot.yml` in the source repository (never in a `-public` mirror that only holds built output):

```yaml
version: 2
updates:
  - package-ecosystem: "npm"
    directory: "/"            # an AstroLog keeps its package in /astro: directory: "/astro"
    schedule:
      interval: "monthly"
    open-pull-requests-limit: 5
    groups:
      minor-and-patch:
        update-types: ["minor", "patch"]
      majors:
        update-types: ["major"]

  - package-ecosystem: "github-actions"
    directory: "/"
    schedule:
      interval: "monthly"
    groups:
      actions:
        patterns: ["*"]
```

Sites with a rehearsal lane: `target-branch: preview`

Where `preview` publishes a rehearsal and `main` publishes live, add `target-branch: "preview"` to each entry. Version updates then rehearse before a Promote PR takes them live.

- **Dependabot reads its config only from the default branch** (`main`). A config change sitting on `preview` does nothing until it's promoted. Until then, PRs keep opening against `main` .
- **`target-branch` applies to version updates only.** Security fixes always open against the default branch. Retarget one before merging: `gh pr edit <n> --base preview` .
- Once the config reaches `main` , Dependabot moves some open PRs itself. Others it leaves on `main` ; retarget or close those.

Commit config-only changes with `[skip ci]`

Pushing `dependabot.yml` to `main` can trigger a deploy: a site rebuild, or a live bot redeploy. Adding `[skip ci]` anywhere in the commit message stops push-triggered workflows for that commit. Dependabot's own first scan still runs.

Holding a major on purpose

A major update can build and still break something nearby. **Example:** TypeScript 7 compiled and type-checked, but `typescript-eslint` didn't support it yet, so `eslint` stopped. Hold it in the config, with the reason where the next person will see it:

```
    ignore:
      # TypeScript 7 builds, but typescript-eslint doesn't support it yet, so `eslint` stops
      # (tested 2026-10-26). Remove this once typescript-eslint supports TS 7.
      - dependency-name: "typescript"
        update-types: ["version-update:semver-major"]
```

- Once the hold reaches the default branch, **Dependabot closes the open PR itself**: *"typescript is no longer being updated by Dependabot"*.
- The PR comment `@dependabot ignore this major version` does the same thing. But it's invisible to the next person, and Dependabot may not answer it on a PR it stopped tracking (for example after a config moved to `preview`). Prefer the config.

Test every PR before merging

Each PR gets its own throwaway copy, beside a baseline built from the branch it targets:

```
git fetch origin "pull/<n>/head:dbt-<n>"
git worktree add ../dbt-<n> dbt-<n>        # and ../dbt-base from origin/main (or preview)
cd ../dbt-<n> && npm ci && npx tsc --noEmit && npx eslint && npm run build
cd - && git worktree remove --force ../dbt-<n> && git branch -D dbt-<n>
```

Compare against the baseline: the same checks green, the same page count, no new console errors. Then **look at what the site produces**:

A build can pass while the output is wrong

An AstroLog's minor-and-patch group took `satori` (it draws the share cards) from 0.18 to 0.63. In 0.x versions a "minor" can break things. The site built cleanly, but the share cards came out with the frame's edges and the footer cut off. Only comparing the images before and after caught it. **Whenever an update touches something that draws (images, PDFs, cards, charts), open a few outputs.**

The cause and the fix:

- **Cause:** satori 0.63 sizes boxes like CSS does (content-box). An element with a set width plus padding or a border grows past its size. 0.18 counted the padding inside (border-box).

- **Fix:** one line in each card generator's style helper makes every element `boxSizing: "border-box"`.
- **Proof:** with that line, 0.18's output stayed byte-identical (231 of 231 cards across the AstroLogs), and 0.63 drew the same layout.
- **Lesson:** fix the code so it works on both versions *before* merging the update, then let the update through.

A broad `npm audit fix` can nest a second copy

On a wallet dApp, `npm audit fix` re-resolved the tree and nested a second `@wagmi/connectors` (8.x) under `@reown/appkit-adapter-wagmi`. The build then failed with *"Package path ./tempo is not exported"*. The cure was to start again from `main`'s lockfile and bump only what was meant:

- the one direct package (`next`)
- transitive patches pinned through `package.json` → `overrides` (`ws`, `uuid`)

Check for a nested copy afterwards: `ls node_modules/<parent>/node_modules`.

Test on the workflow's Node, not just yours

astro 7 needs Node 22.23.4 or newer, but the site's `deploy.yml` still pinned Node 20. It built locally on a newer Node, then would have failed in CI. Before merging a framework major, check the `node-version` in the deploy workflow, and raise it in the same PR if needed.

Pin the long-term release, not a bare major (`node-version: 24`, not `22`). Even a *minor* update can raise the floor: astro 7.4 needs Node 22.23.4 or newer. On 2026-10-26, GitHub's runner resolved `22` to 22.23.3, one patch short. Three sites built fine locally, merged, and then stopped in CI with *"Node.js v22.23.3 is not supported by Astro"*. A failed build publishes nothing, so visitors kept the last good version, and moving the workflows to 24 fixed it. **Test on the exact version CI resolves to.** The setup step's log shows it.

Merge one at a time, then check the live site

A small script does the patient part. Two shapes have been used:

- **Into `preview`**, for sites with a rehearsal lane.
- **Into `main`**, for sites without one.

Both follow the same rules:

1. **List open Dependabot PRs when it starts.** Merge **only the numbers that tested green.** Print held ones with their reason, and anything new as "untested, skipped".
2. **One PR at a time per repository, squash-merged.** If one conflicts (lockfiles often do after a sibling merges), comment `@dependabot rebase` and wait.
3. **After each merge, wait for that repository's own deploy or sync run** on the new commit.
4. **Check a real asset:** fetch the live page, take one stylesheet from its HTML, and request it. A page that returns 200 can still be unstyled, so checking only the page isn't enough.
5. **Any failure stops that repository's queue.** Other repositories carry on, and the live site keeps its last good build.

Order: **security fixes first, then Actions, then minor and patch.** Majors only with a tested reason.

Accepted advisories: write them down

Sometimes the only fix breaks the product. Example: a `zod` and a `decode-uri-component` advisory, buried inside wallet libraries. Their fixes need zod 4 and an ESM-only package, and forcing either breaks the wallet. Both are memory or slowness problems in the visitor's own browser. Record them in the repository's `AGENTS.md`: which advisory, why it's tolerable, what would break, and when to look again. The next person, or AI helper, then won't "fix" it into an outage.

For the AI helper

- **Testing is yours; merging to a live branch is your person's call** unless they've said otherwise. Hand them a tested merge script and the one line that starts it, then watch its log.
- **Never merge an untested PR to make a list shorter.** "Held, with a reason" is a finished state.

- **Security PRs come in bursts** right after alerts are turned on. Re-list before each run.

Suggest a change

Noticed something wrong, or a step that could be kinder? Open an issue or a pull request against `main`, changing this page. A person with the role reads it and merges with a merge commit, so your name stays on it. An AI helper may do all of that up to the merge. Not on GitHub? Tell any steward you know, or your helper, in your own words.

Part of the walkthroughs. Official docs: Dependabot options reference.